

THE ANATOMY OF A PHONE NUMBER

Packet-Level Telecommunications

VOLUME III OF III

SIP, SDP, GTP-U, RTP and the 5G radio stack, field by field

The same call at packet granularity: SIP and SDP field by field, GTP-U tunnelling, PDCP/RLC/MAC/PHY, the RTP header, jitter and loss, the control-plane stack, and the point at which the telephone number stops existing on the wire.



A LAYERED STUDY OF TELEPHONE NUMBERING, MOBILE NETWORKS AND PACKET TELEPHONY

PART III / III

About this volume

The same call at packet granularity: SIP and SDP field by field, GTP-U tunnelling, PDCP/RLC/MAC/PHY, the RTP header, jitter and loss, the control-plane stack, and the point at which the telephone number stops existing on the wire.

This is one of three volumes. Each descends a level further from the digits a human types toward the electromagnetic wave that eventually carries the speech. The volumes are meant to be read in order, but each is self-contained.

VOLUME I

The Architecture of a Phone Number

VOLUME II

Anatomy of a 5G Voice Call

VOLUME III

Packet-Level Telecommunications

Diagrams are numbered by chapter. A list of figures follows the contents, so any diagram can be found directly.

Contents

CH 1	The Packet Stack	8
1.1	First, establish the packet-stack picture	8
CH 2	Dissecting a SIP INVITE	10
2.1	The first packet we care about: SIP INVITE	10
2.2	Let's dissect it field by field	10
2.3	Via	11
2.4	From	12
2.5	To	12
2.6	Call-ID	13
2.7	CSeq	14
CH 3	SDP and the Media Description	15
3.1	The SDP payload	15
3.2	What does SDP actually mean?	15

3.3 SIP doesn't carry your voice	17
CH 4 From SIP to GTP-U	18
4.1 What happens to the SIP packet?	18
4.2 Now we reach GTP-U	19
4.3 Why does the operator need tunneling?	20
4.4 GTP-U conceptual packet	20
4.5 The GTP-U tunnel identifier	21
CH 5 Down the Radio Stack	23
5.1 Now the packet reaches the gNB	23
5.2 PDCP	23
5.3 RLC	24
5.4 MAC	25
5.5 PHY	25
CH 6 Where the Phone Number Disappears	27
6.1 The phone number disappears early	27
CH 7 RTP in Detail	29
7.1 Now let's inspect RTP	29
7.2 RTP Version	30
7.3 Payload Type	30
7.4 Sequence number	31
7.5 Timestamp	32
7.6 SSRC	32
7.7 The actual RTP payload	33
7.8 RTP over UDP	33
CH 8 Impairments: Loss, Jitter and RTCP	35
8.1 What does packet loss sound like?	35
8.2 What does jitter do?	36
8.3 Jitter buffer	36
8.4 RTCP	38
CH 9 Encapsulation, Segmentation and the Control Plane	40
9.1 Now let's return to the 5G radio	40

9.2 Encapsulation vs segmentation	41
9.3 The control-plane packet is different	42
9.4 RRC	43
9.5 NGAP	44
9.6 SCTP	45
CH 10 Service-Based Architecture and HTTP/2	47
10.1 The 5G core itself is service-oriented	47
10.2 Where HTTP/2 appears	47
CH 11 Comparing the Layers	49
11.1 Now compare the layers	49
11.2 Notice the progressive loss of meaning	49
CH 12 One Packet's Journey	51
12.1 One packet's journey	51
12.2 What the router actually sees	53
12.3 Why this architecture scales	54
12.4 And this explains number portability	55
CH 13 Reading It in Wireshark	57
13.1 What you should eventually be able to read in Wireshark	57
13.2 The packet-capture hierarchy	57
13.3 And here is the critical security boundary	59
CH 14 The Final Mental Model	60
14.1 The final mental model	60

Figures

Fig. 1.1 APPLICATION → IP → Rf waveform	8
Fig. 1.2 SIP → IP → 5g ran	9
Fig. 1.3 NAS → NGAP → IP	9
Fig. 2.1 INVITE → SIP/2.0	11
Fig. 2.2 Via → response routing	11
Fig. 2.3 Via → That distinction matters	12
Fig. 2.4 actual IP address → It isn't	13

Fig. 2.5	telephone number → "which call?"	13
Fig. 2.6	Sip dialog → Invite / update / bye	14
Fig. 3.1	m=audio → 96	16
Fig. 3.2	payload type 96 → 16000 Hz RTP clock	16
Fig. 4.1	SIP message → IP packet → radio protocol stack	18
Fig. 4.2	Encapsulation stack: IP header over SIP message	19
Fig. 4.3	Original packet → UPF	19
Fig. 4.4	UPF and its constituent parts	20
Fig. 4.5	Outer IP header → tcp/udp / → This is encapsulation	21
Fig. 4.6	Outer IP → TEID → inner IP packet	22
Fig. 5.1	PDCP → PHY	23
Fig. 5.2	IP packet → Pdcip pdu	24
Fig. 5.3	Pdcip pdu → RLC	24
Fig. 5.4	RLC → MAC	25
Fig. 5.5	MAC → modulation → antenna	26
Fig. 6.1	+9198BBBBBB20 → destination session → electromagnetic waveform	27
Fig. 6.2	www.example.com → IP → radio	28
Fig. 7.1	0 1 / 2 3 → timestamp / → audio payload ... /	30
Fig. 7.2	RTP session and its constituent parts	32
Fig. 7.3	Ethernet/IP → UDP → codec payload	33
Fig. 7.4	Encapsulation stack: IP header over encoded speech	34
Fig. 8.1	incoming RTP → ordered audio → speaker	37
Fig. 8.2	larger buffer → smaller buffer → more sensitivity to jitter	38
Fig. 9.1	[PDCP] → [PHY]	40
Fig. 9.2	IP → Rtp + udp + ip	41
Fig. 9.3	large PDU → fragment 3	41
Fig. 9.4	NAS → NGAP → Different state machine	43
Fig. 9.5	UE → gNB	44
Fig. 9.6	gNB → AMF	44
Fig. 9.7	RAN side → 5GC	45
Fig. 9.8	NGAP → transport network	45
Fig. 9.9	Encapsulation stack: NGAP over Ethernet/etc.	46
Fig. 9.10	Encapsulation stack: GTP-U over IP	46

Fig. 10.1	NRF → PCF	47
Fig. 10.2	AMF → UDM	48
Fig. 10.3	SMF → PCF	48
Fig. 11.1	SEMANTIC → NETWORK → PHYSICAL	50
Fig. 12.1	PDCCP → PHY	52
Fig. 12.2	RF → IP → speaker	53
Fig. 12.3	number → mobility state → radio resources	55
Fig. 12.4	+91 98bbbbbb20 → Operator B	56
Fig. 13.1	GTP-U → RTP	58
Fig. 13.2	SIM authentication → IPsec/TLS/etc → media security	59
Fig. 14.1	Phone number → other phone	60
Fig. 14.2	Human intent → gNB → EAR	61
Fig. 14.3	5g control plane → Smf / udm / ausf → UPF	62

Introduction

Good. Now we stop treating the network as boxes with mysterious arrows and inspect the actual protocol objects.

One caveat first: there is no single universal byte-for-byte packet trace for every 5G voice call. Operators differ in IMS topology, security, NAT/interconnect, codecs, QoS, and even whether media is anchored through a media function. So I'll build a representative 5G SA VoNR call and distinguish standardized fields from illustrative values.

The relevant specifications are 3GPP TS 24.229 for IMS SIP/SDP, TS 38.331 for NR RRC, and related 3GPP specifications for the 5G radio/user plane. [3GPP Portal](#)

The Packet Stack

1.1 First, establish the packet-stack picture

1.1 First, establish the packet-stack picture

When Alice says:

Hello

there isn't one packet called "Hello."

The data gets encapsulated through multiple layers.

Conceptually:

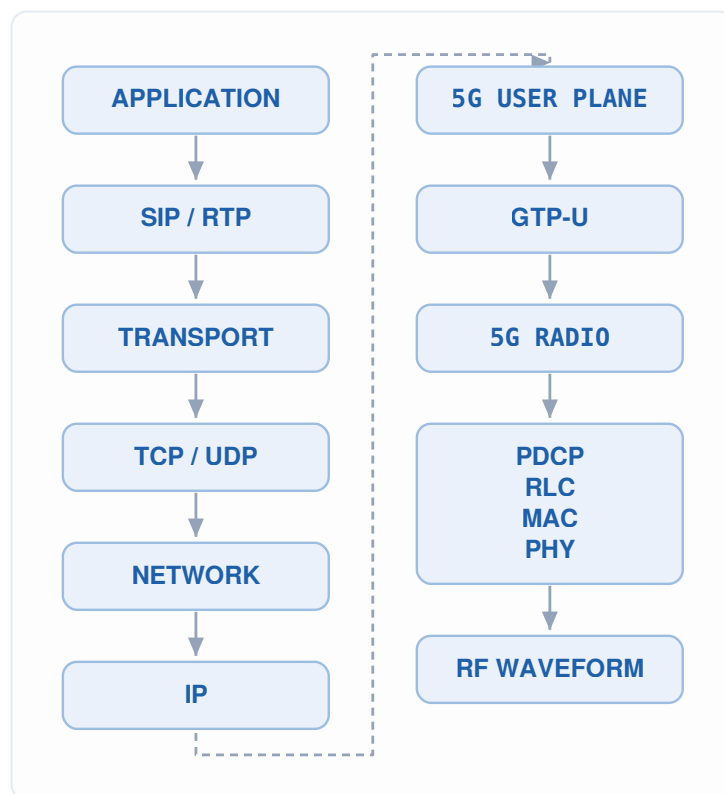


Figure 1.1 APPLICATION → IP → Rf waveform

And that's only the user plane.

Call establishment has another stack:

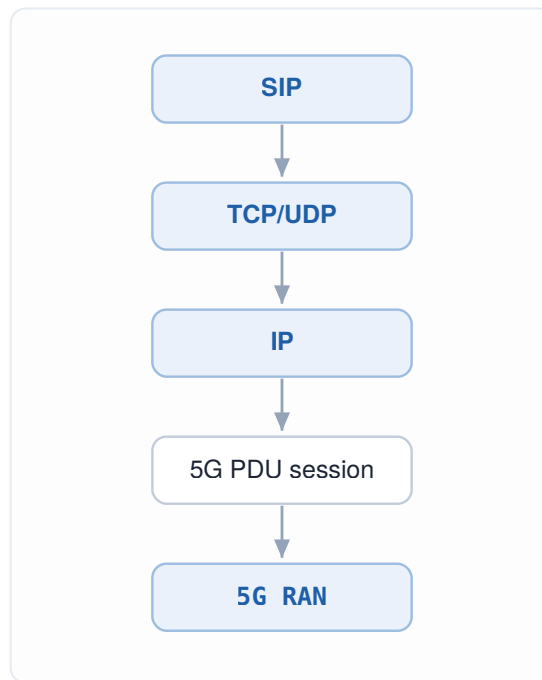


Figure 1.2 SIP → IP → 5g ran

while 5G control signaling has things like:

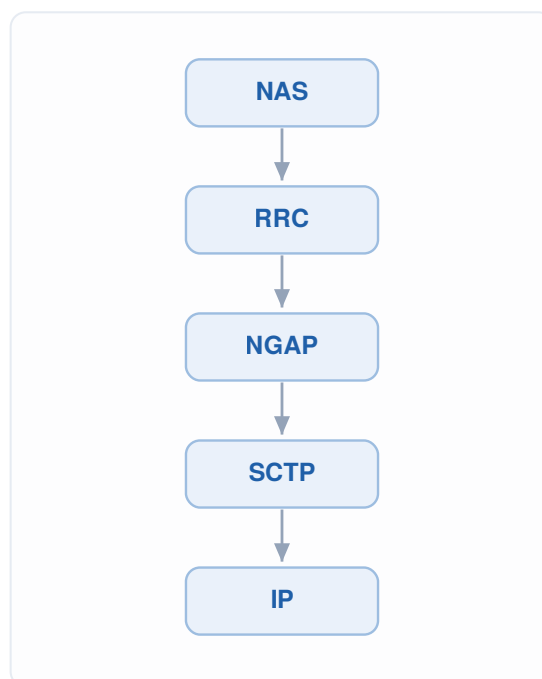


Figure 1.3 NAS → NGAP → IP

So already we have multiple protocol stacks operating simultaneously.

Dissecting a SIP INVITE

2.1 The first packet we care about: SIP INVITE

2.2 Let's dissect it field by field

2.3 Via

2.4 From

2.5 To

2.6 Call-ID

2.7 CSeq

2.1 The first packet we care about: SIP INVITE

Let's begin at the application layer.

Alice wants:

```
+91 98BBBBBB20
```

The IMS stack constructs something conceptually similar to:

INVITE sip:+9198BBBBBB20@example-ims.operator.com SIP/2.0

Via: SIP/2.0/TCP ...

Max-Forwards: 70

From: <sip:+9198AAAAAA10@example-ims.operator.com>;tag=abc123

- To: <sip:+9198BBBBBB20@example-ims.operator.com>
- Call-ID: 837462@example-ims.operator.com
- CSeq: 1 INVITE
- Contact: <sip:...>
- Content-Type: application/sdp

Content-Length: ...

This is not hypothetical protocol design. SIP's standardized INVITE mechanism and headers such as Via, From, To, Call-ID, CSeq, Contact, and SDP body are defined in RFC 3261 and IMS-specific behavior is standardized by 3GPP TS 24.229. RFC Editor

2.2 Let's dissect it field by field

Take:

INVITE sip:+9198BBBBBB20@example-ims.operator.com SIP/2.0

Three major components:

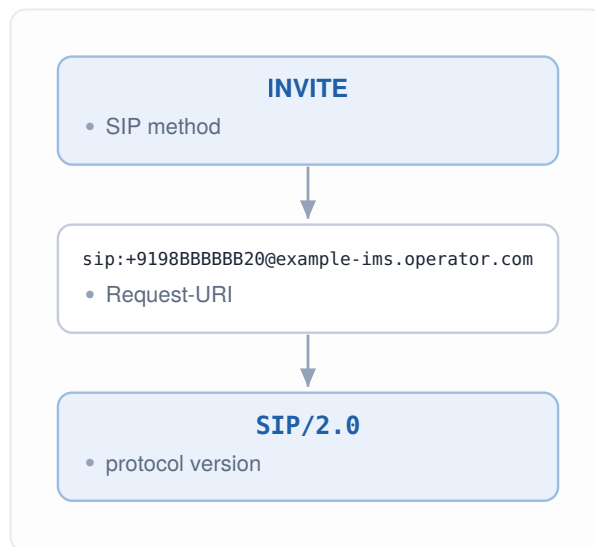


Figure 2.1 INVITE → SIP/2.0

INVITE means:

establish a session.

SIP is deliberately a signaling protocol, not the protocol carrying the actual speech. RFC 3261 describes SIP as an application-layer control protocol for creating, modifying and terminating sessions. RFC Editor

2.3 Via

Example:

Via: SIP/2.0/TCP pcscf.operator.com;branch=z9hG4bKabc123

This tells downstream SIP entities where responses should be sent and contains transaction-routing information.

The important idea:

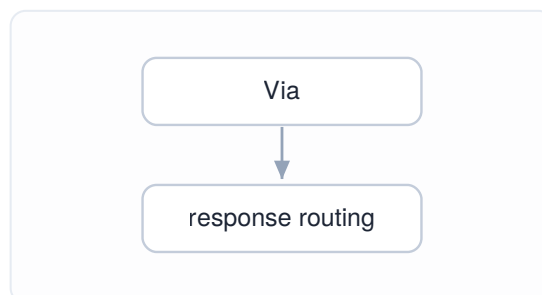


Figure 2.2 Via → response routing

It is not:

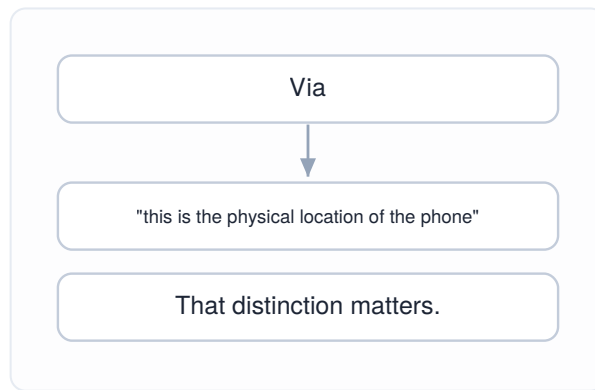


Figure 2.3 Via → That distinction matters

RFC 3261 specifically defines the Via header and its branch parameter for transaction identification.

2.4 From

From: <sip:+9198AAAAAA10@example-ims.operator.com>;tag=abc123

This represents the logical originator.

Notice:

+91 98AAAAAA10

is represented inside an IMS/SIP identity.

We're converting:

telephone namespace

into:

IMS namespace

This is one of the layers we discussed earlier.

2.5 To

To: <sip:+9198BBBBBB20@example-ims.operator.com>

This identifies the intended destination.

But don't make the mistake:

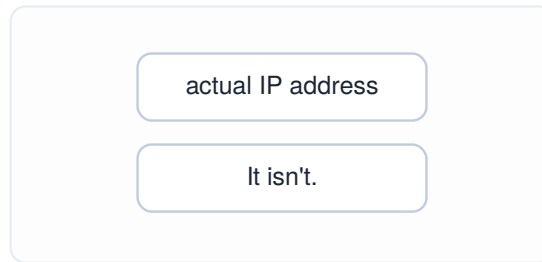


Figure 2.4 actual IP address → It isn't

It is a logical destination.

SIP routing machinery determines where the request ultimately goes.

2.6 Call-ID

Call-ID: 837462@example-ims.operator.com

This identifies the SIP dialog/session context.

You can think:

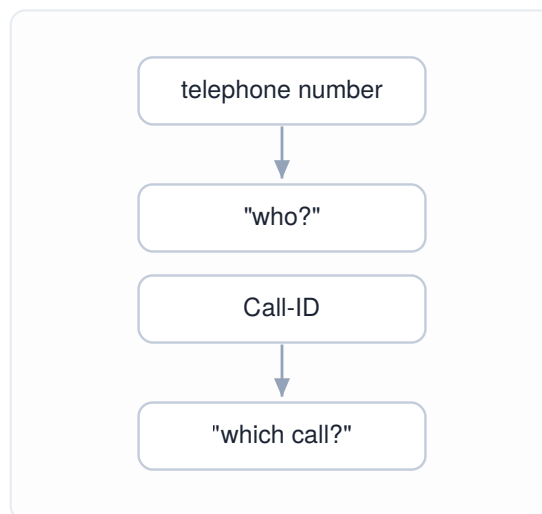


Figure 2.5 telephone number → "which call?"

Suppose Alice calls Bob twice:

- Call #1
- Call-ID = AAAAA
- Call #2
- Call-ID = BBBBB

Same people.

Different SIP dialogs.

This is a classic example of layered identity.

2.7 CSeq

CSeq: 1 INVITE

This provides sequencing for SIP requests within a dialog.

Later:

CSeq: 2 BYE

might terminate the call.

So the SIP dialog has its own state machine.

You can visualize:

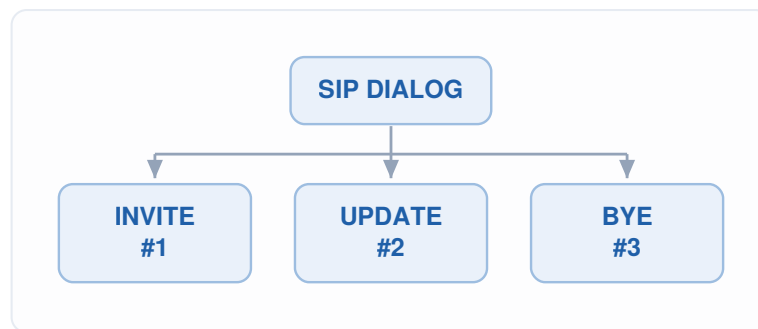


Figure 2.6 Sip dialog → Invite / update / bye

The network isn't just shoving messages around. It maintains protocol state.

CHAPTER 3

SDP and the Media Description

3.1 The SDP payload

3.3 SIP doesn't carry your voice

3.2 What does SDP actually mean?

3.1 The SDP payload

Now the interesting part.

The SIP INVITE may contain:

Content-Type: application/sdp

followed by something conceptually like:

```
v=0
o=- 123456 123456 IN IP4 192.0.2.10
s=-
c=IN IP4 192.0.2.10
t=0 0
m=audio 40000 RTP/AVP 96
a=rtpmap:96 EVS/16000
a=sendrecv
```

SDP uses fields such as:

The formal SDP grammar and session/media fields are specified by RFC 4566. [RFC Editor](#)

3.2 What does SDP actually mean?

Take:

```
m=audio 40000 RTP/AVP 96
```

Break it down:

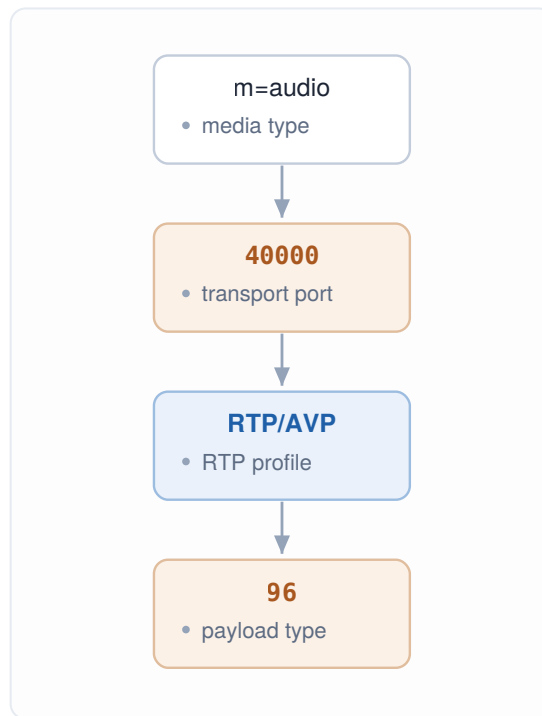


Figure 3.1 m=audio → 96

Then:

```
a=rtpmap:96 EVS/16000
```

means approximately:

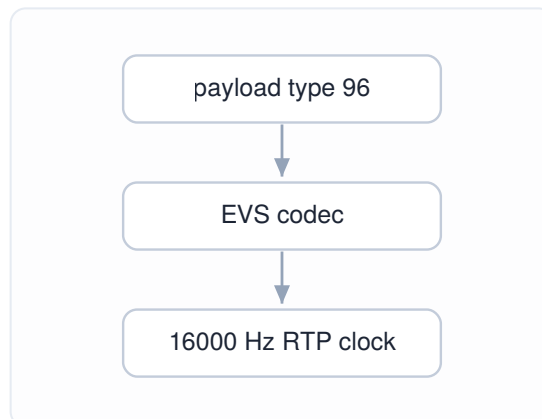


Figure 3.2 payload type 96 → 16000 Hz RTP clock

So SDP is essentially negotiating:

"Here is how I can receive/send the media."

3.3 SIP doesn't carry your voice

This deserves a giant conceptual boundary:

- | | | |
|---|----------------|---|
| • SIP | • CALL CONTROL | •  |
| •  | | • ACTUAL MEDIA |

SIP:

```
INVITE
180 Ringing
200 OK
ACK
BYE
```

RTP:

- | | | |
|---------|---------|---------|
| • audio | • audio | • audio |
| • audio | • audio | |

RFC 3261 explicitly describes SDP carried inside SIP invitations as a means for participants to agree on compatible media. RFC Editor

From SIP to GTP-U

4.1 What happens to the SIP packet?

4.2 Now we reach GTP-U

4.3 Why does the operator need tunneling?

4.4 GTP-U conceptual packet

4.5 The GTP-U tunnel identifier

4.1 What happens to the SIP packet?

The application generates the SIP message.

Then transport/network layers encapsulate it.

Conceptually:

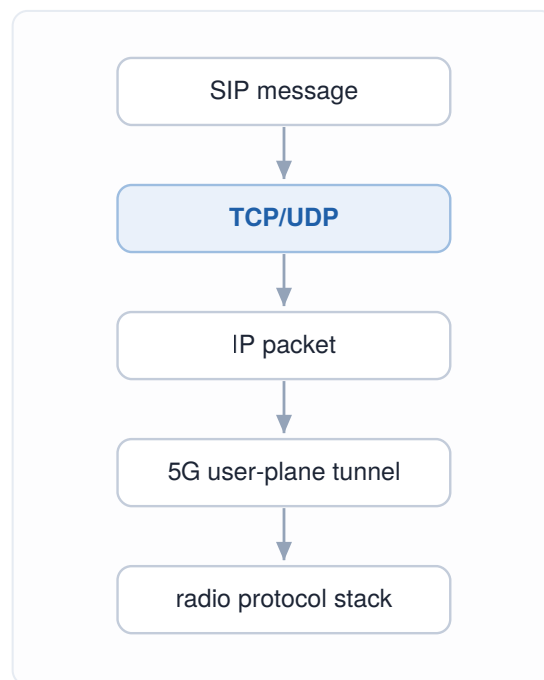


Figure 4.1 SIP message → IP packet → radio protocol stack

At Alice's phone:

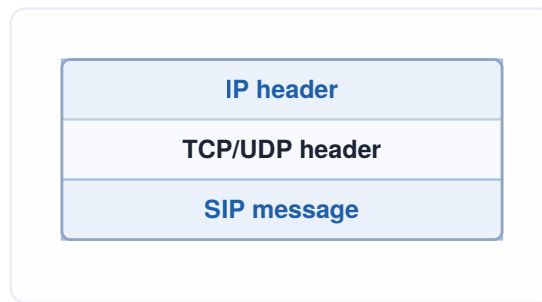


Figure 4.2 Encapsulation stack: IP header over SIP message

Then 5G networking adds additional machinery around the packet as it crosses the radio/ user-plane infrastructure.

4.2 Now we reach GTP-U

Suppose Alice's IP packet is:

Source IP:

10.20.30.40

Destination IP:

10.50.60.70

The operator's 5G network may transport this using:

GTP-U

GTP-U means:

GPRS Tunnelling Protocol - User Plane

The basic idea:

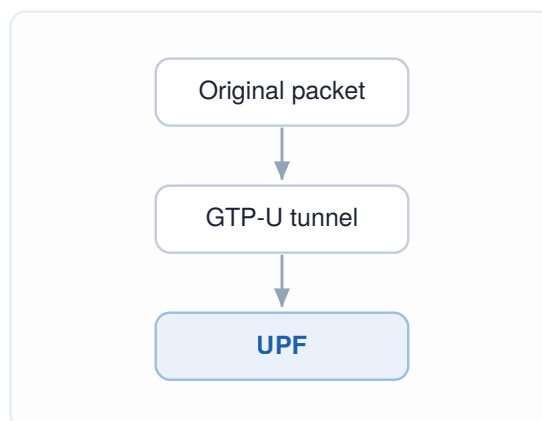


Figure 4.3 Original packet → UPF

You can think of GTP-U as:

"Carry this subscriber's original IP packet through the mobile operator's transport network."

4.3 Why does the operator need tunneling?

Because the operator has thousands or millions of subscribers.

Imagine:

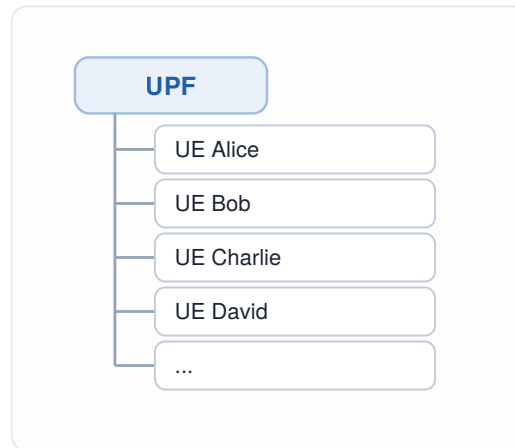


Figure 4.4 UPF and its constituent parts

The network needs to maintain logical association between packets and subscriber sessions.

The mobile core therefore uses tunnel/context mechanisms rather than treating every phone as if it were directly connected to the Internet.

4.4 GTP-U conceptual packet

A simplified GTP-U packet looks like:

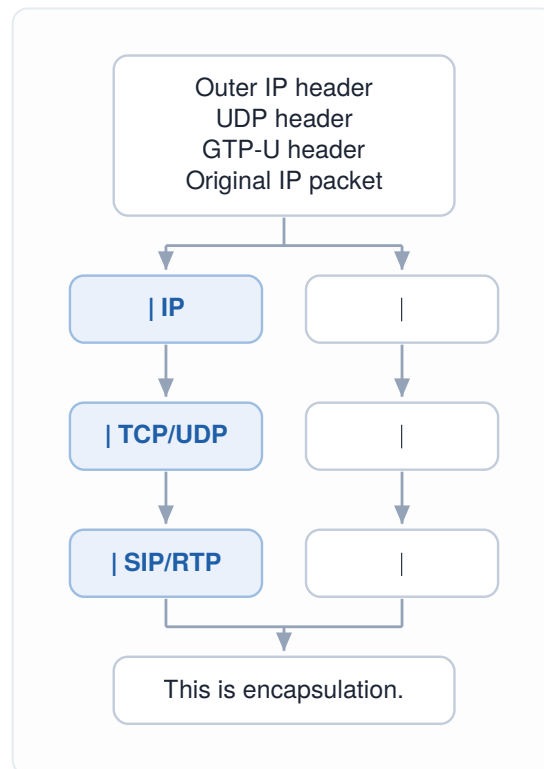


Figure 4.5 Outer IP header → | tcp/udp / | → This is encapsulation

The operator isn't necessarily modifying the original application payload. It can wrap it in a tunnel.

4.5 The GTP-U tunnel identifier

GTP-U uses a:

TEID

Tunnel Endpoint Identifier.

Conceptually:

TEID = 0x8A72F1C3

This tells the receiving GTP-U endpoint which tunnel/context the packet belongs to.

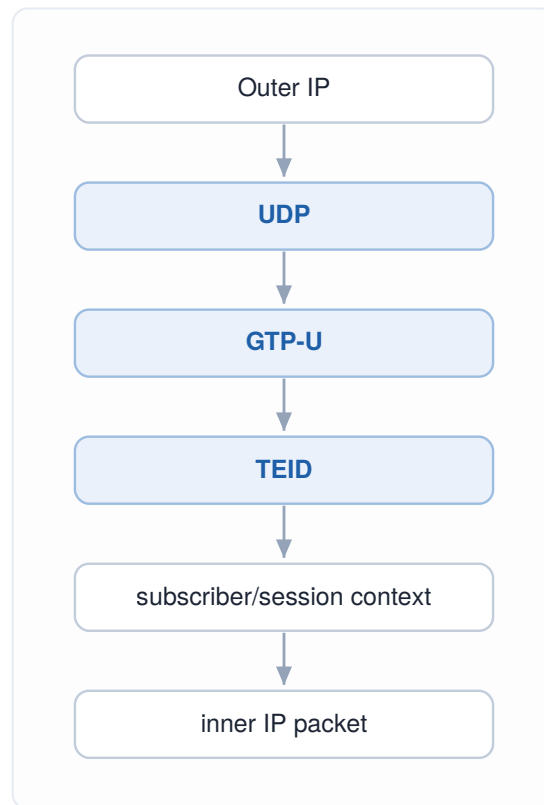


Figure 4.6 Outer IP → TEID → inner IP packet

This is one of the most important concepts in mobile packet networking.

Down the Radio Stack

5.1 Now the packet reaches the gNB

5.2 PDCP

5.3 RLC

5.4 MAC

5.5 PHY

5.1 Now the packet reaches the gNB

The gNB receives user-plane traffic through the 5G radio stack.

But the radio doesn't simply transmit the entire Ethernet/IP packet as one giant blob.

The packet is processed through:

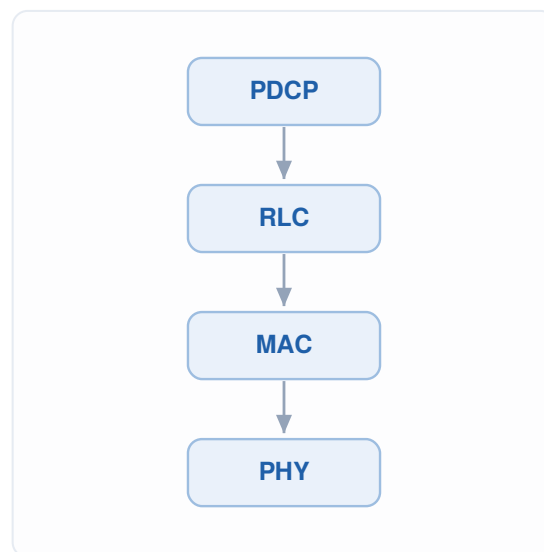


Figure 5.1 PDCP → PHY

3GPP maintains separate specifications for NR RRC, RLC and PDCP, illustrating that these are distinct protocol layers rather than one monolithic "5G protocol."

5.2 PDCP

PDCP sits above RLC.

It handles functions including:

- ciphering
- integrity protection
- sequence numbering
- header compression
- reordering

depending on the bearer/configuration.

Conceptually:

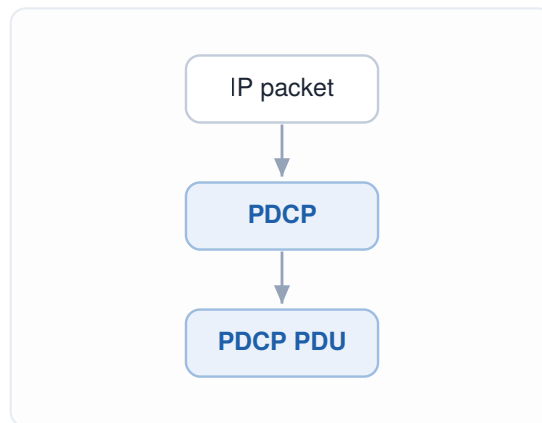


Figure 5.2 IP packet → Pdcp pdu

The actual transmitted bytes are now becoming progressively further removed from the original SIP/RTP structure.

5.3 RLC

RLC can segment and reassemble higher-layer data.

Imagine your packet is:

[AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA]

but the radio transmission can only conveniently carry:

- [AAAA]
- [AAAA]
- [AAAA]
- [AAAA]

RLC can perform segmentation/reassembly depending on configuration.

Conceptually:

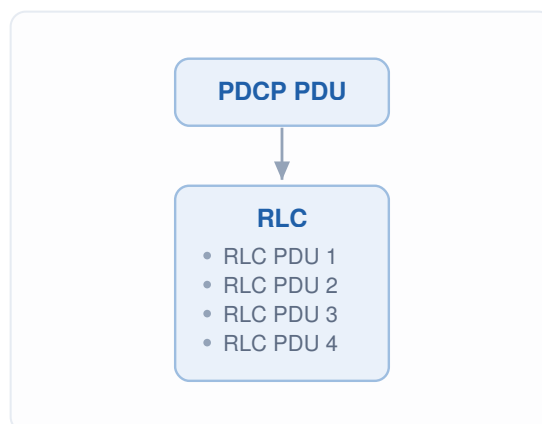


Figure 5.3 Pdcp pdu → RLC

5.4 MAC

MAC receives data from RLC and interacts with the scheduler.

Conceptually:

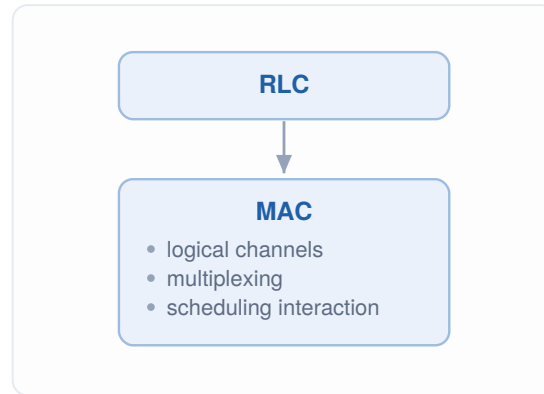


Figure 5.4 RLC → MAC

Now the network has to answer:

Which UE gets which radio resources at which instant?

This is a completely different problem from:

Which subscriber does this phone number belong to?

That separation is why layered architecture exists.

5.5 PHY

Finally:

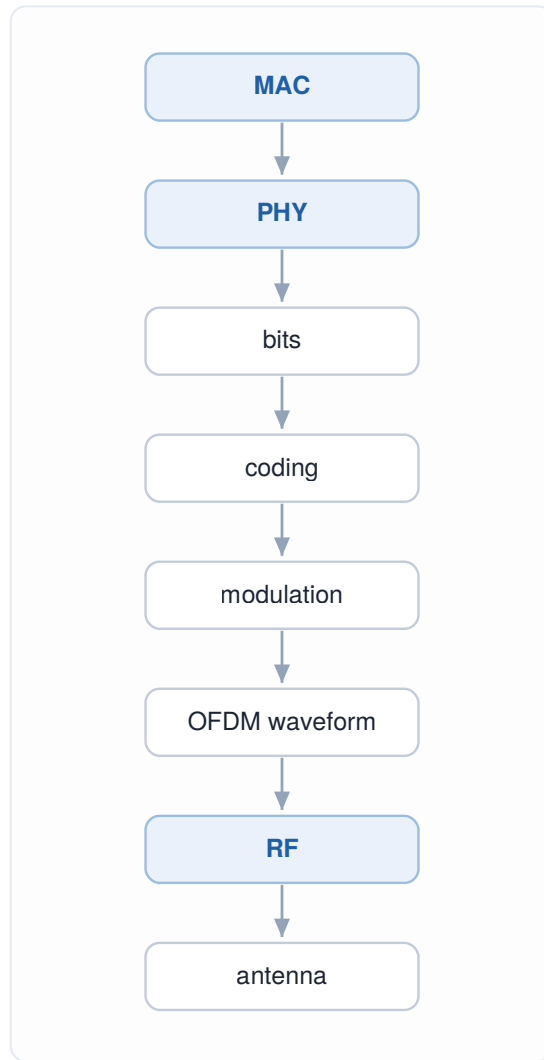


Figure 5.5 MAC → modulation → antenna

The digital packet has become an electromagnetic waveform.

At this point:

+91 98BBBBBB20

is nowhere in the physical signal as a literal ASCII string.

The number was consumed much earlier by the signaling/routing machinery.

This is an important conceptual endpoint.

Where the Phone Number Disappears

6.1 The phone number disappears early

6.1 The phone number disappears early

Trace the information:

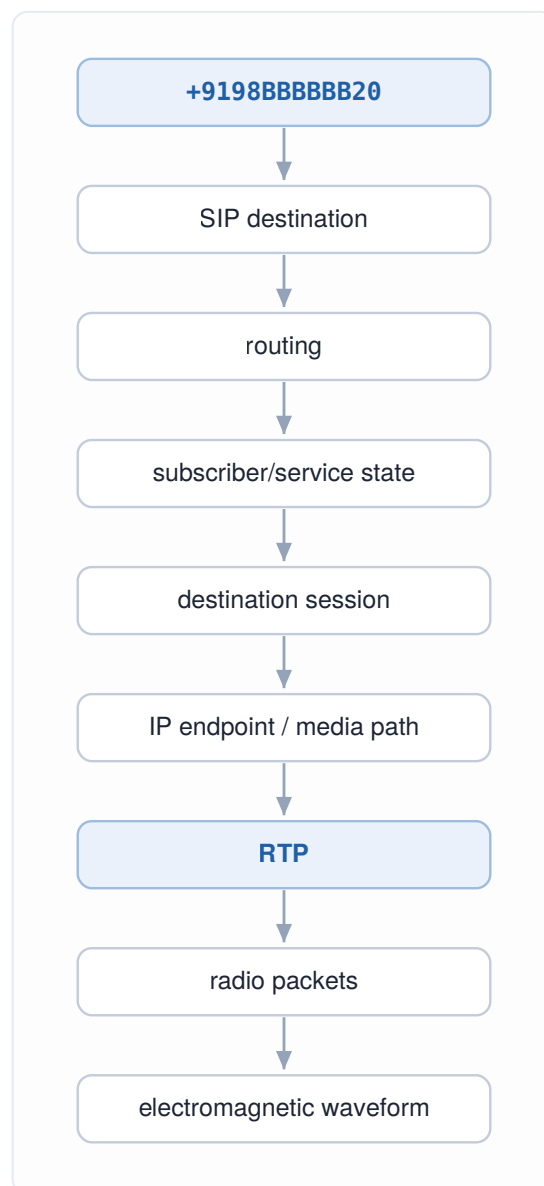


Figure 6.1 +9198BBBBBB20 → destination session → electromagnetic waveform

The literal phone number doesn't need to travel through every layer.

It is resolved into other identities.

That's the same principle as:

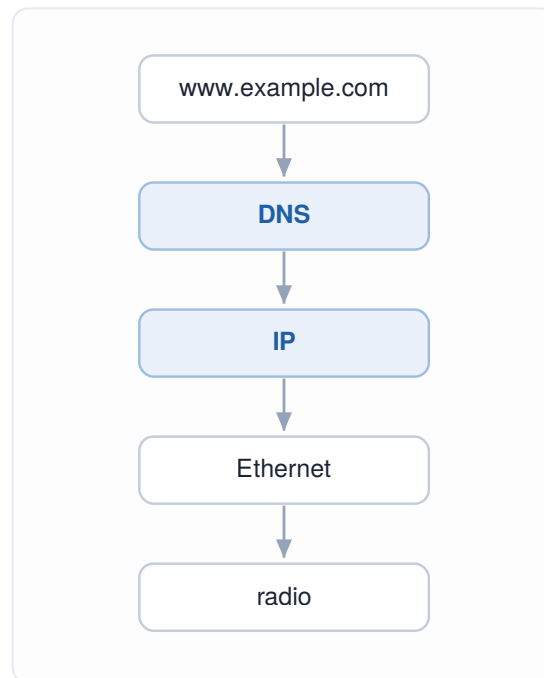


Figure 6.2 `www.example.com` → IP → radio

The original identifier isn't carried through every layer.

RTP in Detail

7.1 Now let's inspect RTP

7.2 RTP Version

7.3 Payload Type

7.4 Sequence number

7.5 Timestamp

7.6 SSRC

7.7 The actual RTP payload

7.8 RTP over UDP

7.1 Now let's inspect RTP

Suppose Alice says:

hello

The codec creates an encoded audio frame.

That becomes an RTP packet.

The RTP fixed header is at least 12 bytes and contains fields including:

- V
- P
- X
- CC
- M
- PT
- Sequence Number
- Timestamp
- SSRC
- RFC 3550 defines this structure. [RFC Editor](#)

Visualized:

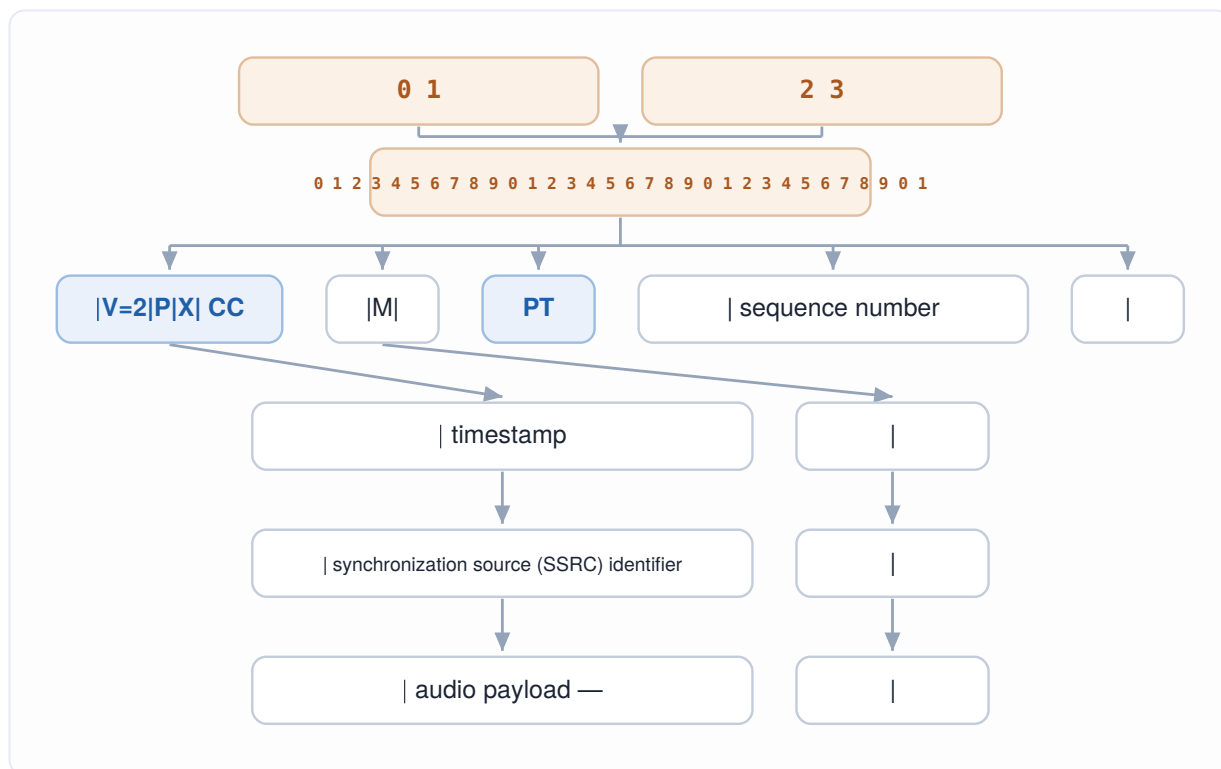


Figure 7.1 0 1 / 2 3 → | timestamp / | → | audio payload ... / |

RFC 3550 specifies RTP as a real-time transport protocol providing payload identification, sequence numbering, timestamping and delivery monitoring. *RFC Editor*

7.2 RTP Version

First:

V = 2

That's:

2 bits

Current RTP is version 2.

So the first two bits are:

10

Binary.

7.3 Payload Type

Suppose:

PT = 96

That means:

payload type 96

For dynamic payload types, the exact codec mapping is established through signaling such as SDP.

PT=96

alone does not universally mean:

EVS

The session configuration tells the endpoints what it means.

This is why SDP and RTP cooperate.

7.4 Sequence number

Suppose:

Sequence = 49100

Next:

49101

Then:

49102

and so on.

This allows the receiver to detect:

- packet missing
- packet reordered
- packet duplicated

Example:

Received:

49100
49101
49103
49104

The receiver notices:

49102 missing

RTP itself does not magically retransmit it. Real-time media often prefers continuing playback over waiting for a late packet.

That's a major difference from reliable TCP-style delivery.

7.5 Timestamp

Suppose:

Timestamp = 38192000

The RTP timestamp represents the media sampling timeline.

It's not simply:

"current Unix time"

It operates according to the media clock.

For audio, the RTP clock rate depends on the payload format.

RFC 3550 specifies the timestamp as representing the sampling instant and explains its use in synchronization and jitter calculations. [RFC Editor](#)

7.6 SSRC

Suppose:

SSRC = 0x7A82F391

This identifies the synchronization source.

Think:

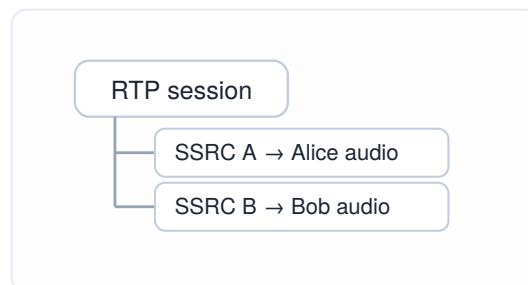


Figure 7.2 RTP session and its constituent parts

RFC 3550 defines SSRC as a 32-bit synchronization-source identifier independent of network address. [RFC Editor](#)

Again:

IP address $\neq$ SSRC

Different abstraction.

7.7 The actual RTP payload

After the 12-byte minimum RTP header:

RTP header – 12+ bytes – codec payload – encoded speech

The payload might contain an encoded speech frame.

Not:

ASCII "hello"

The codec has transformed speech into a compressed representation.

7.8 RTP over UDP

A common stack is:

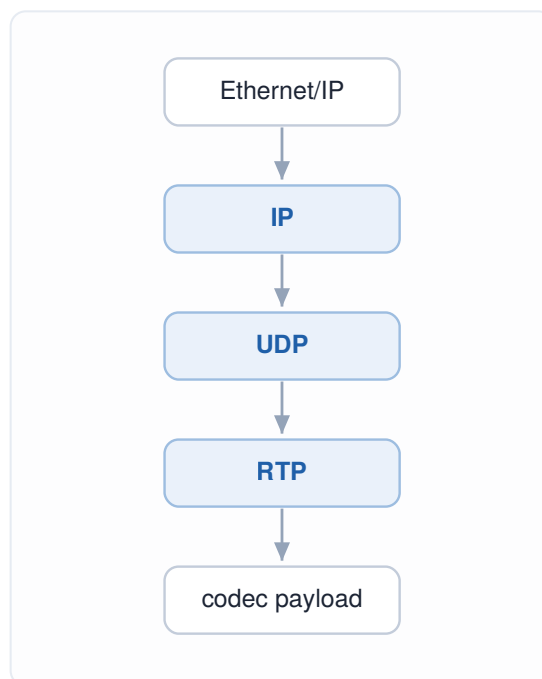


Figure 7.3 Ethernet/IP → UDP → codec payload

So conceptually:

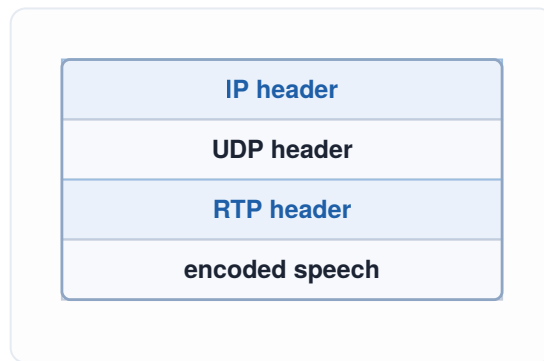


Figure 7.4 Encapsulation stack: IP header over encoded speech

UDP is useful because real-time media generally doesn't want TCP's retransmission semantics.

If a packet arrives 500 ms late:

TCP:

"Here it is!"

Voice:

"Thanks, we're already on another sentence."

Impairments: Loss, Jitter and RTCP

8.1 What does packet loss sound like?

8.2 What does jitter do?

8.3 Jitter buffer

8.4 RTCP

8.1 What does packet loss sound like?

Suppose:

RTP sequence:

1000
1001
1002
1003
1005
1006

Packet:

1004

is missing.

The receiver can use concealment mechanisms.

Instead of:

HELLO

becoming:

HEL_0

the codec may attempt:

HELLO

using packet-loss concealment.

This is why voice can remain intelligible despite packet loss.

8.2 What does jitter do?

Suppose packets are generated evenly:

P1 —10ms— P2 —10ms— P3 —10ms— P4

but arrive:

P1
7ms
P2
19ms
P3
4ms
P4

The receiver experiences:

jitter

RTP/RTCP provide mechanisms for monitoring timing and packet delivery. RFC 3550 specifies interarrival-jitter reporting and RTP timing fields. [RFC Editor](#)

8.3 Jitter buffer

The phone therefore doesn't necessarily play every packet immediately.

Instead:

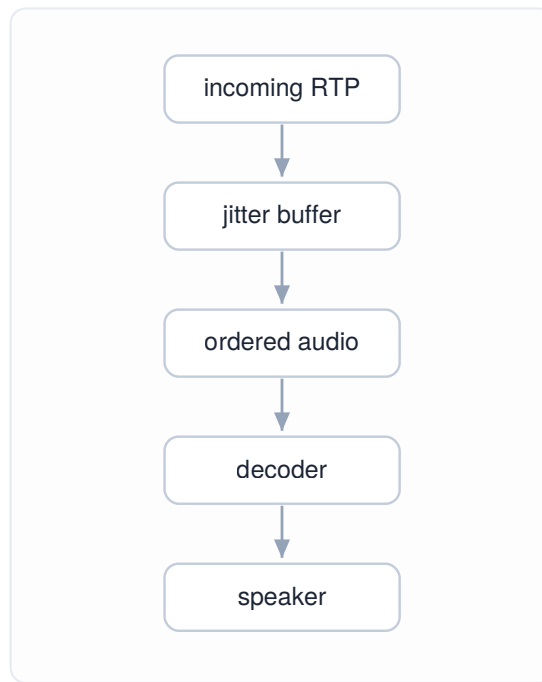


Figure 8.1 incoming RTP → ordered audio → speaker

The buffer deliberately introduces a little latency.

For example:

Network:

P1
P2
P3
P4
P5

The jitter buffer attempts to smooth that out.

Trade-off:

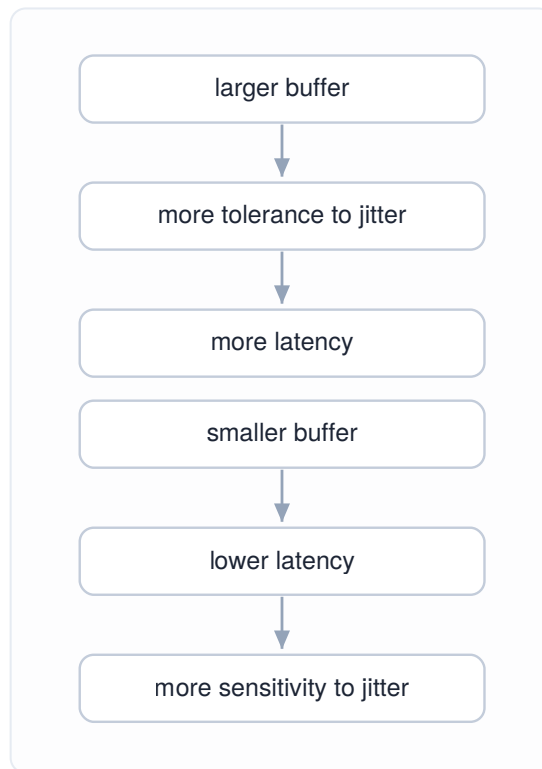


Figure 8.2 larger buffer → smaller buffer → more sensitivity to jitter

Engineering is mostly choosing which failure mode you can tolerate.

8.4 RTCP

RTP has an associated control protocol:

RTCP

It can report things such as:

- packet loss
- jitter
- sender statistics
- receiver reports

For example:

RTCP Receiver Report

SSRC: 0x7A82F391

fraction lost: ...

cumulative lost: ...

highest sequence: ...

jitter: ...

RFC 3550 specifies RTCP sender and receiver reports and the associated statistics. RFC Edit-
or

This allows the system to understand media quality rather than blindly throwing packets into the void.

Encapsulation, Segmentation and the Control Plane

9.1 Now let's return to the 5G radio

9.2 Encapsulation vs segmentation

9.3 The control-plane packet is different

9.4 RRC

9.5 NGAP

9.6 SCTP

9.1 Now let's return to the 5G radio

Here's where things become fascinating.

The RTP packet might conceptually begin as:

[RTP][audio]

Then:

[UDP][RTP][audio]

Then:

[IP][UDP][RTP][audio]

Then mobile user-plane encapsulation:

[GTP-U][IP][UDP][RTP][audio]

Then radio protocols operate around that data:

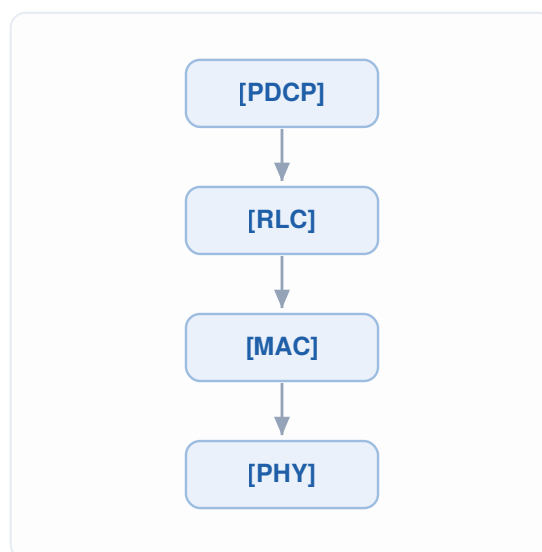


Figure 9.1 [PDCP] → [PHY]

So the same original voice information exists inside increasingly lower-level representations.

9.2 Encapsulation vs segmentation

Don't confuse these.

Encapsulation

Adds protocol headers:

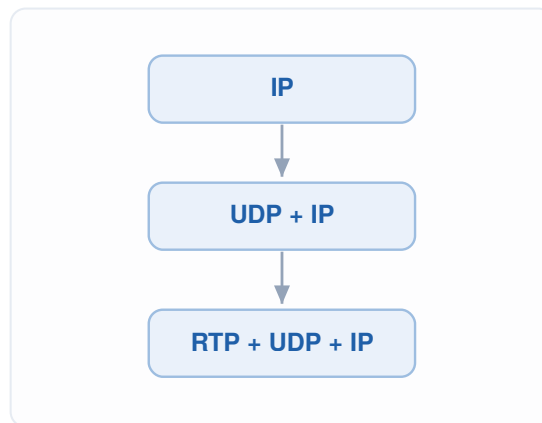


Figure 9.2 IP → Rtp + udp + ip

Segmentation

Splits one higher-layer object into smaller pieces:

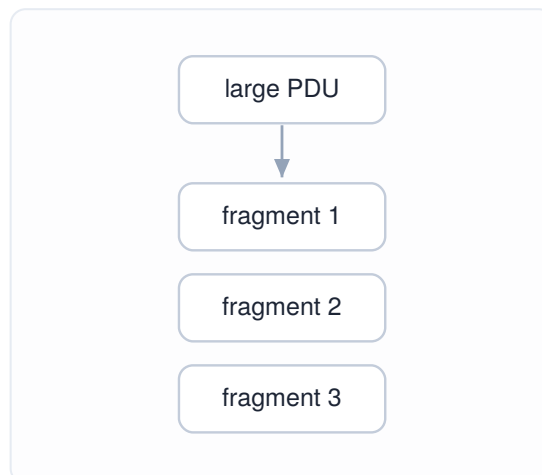


Figure 9.3 large PDU → fragment 3

5G radio protocols can perform segmentation/reassembly as required.

encapsulation ≠ segmentation

Both can happen.

9.3 The control-plane packet is different

Now let's inspect something completely different.

Suppose the UE needs to perform an operation such as:
registration

You may encounter:



NAS

The NAS message is not RTP.

It isn't SIP.

It isn't user-plane traffic.

It is control signaling.

Conceptually:

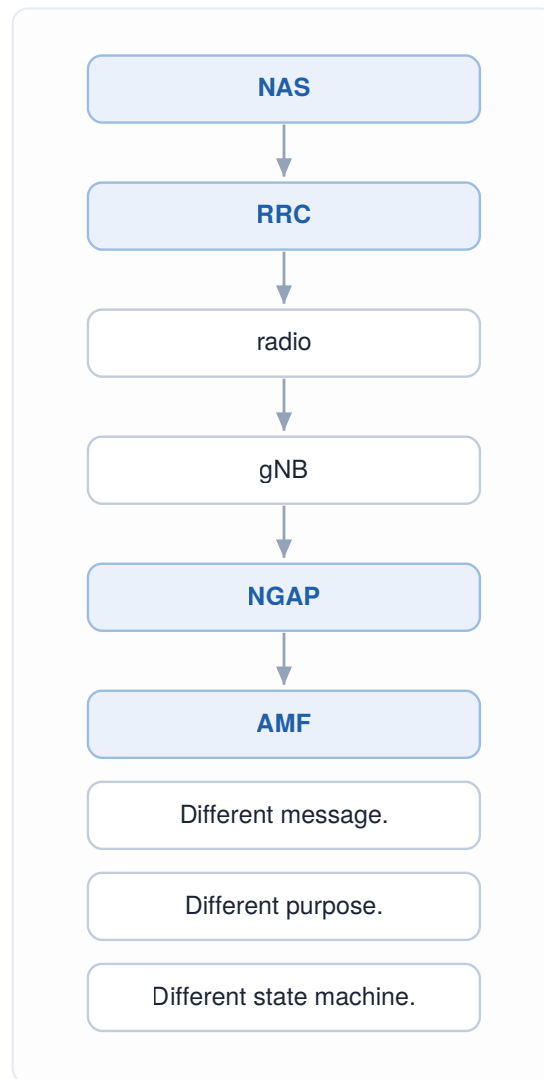


Figure 9.4 NAS → NGAP → Different state machine

9.4 RRC

RRC means:

Radio Resource Control

3GPP TS 38.331 defines the NR RRC protocol.

RRC deals with radio-side control such as:

- connection establishment
- configuration
- measurement
- mobility-related signaling
- radio bearer configuration
- security activation

Conceptually:

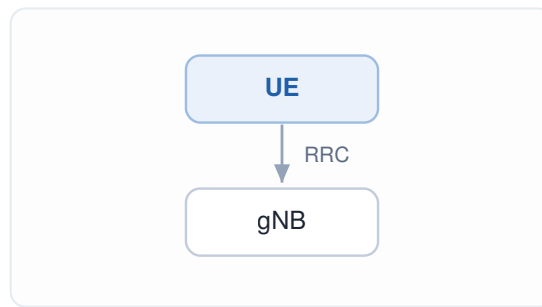


Figure 9.5 UE → gNB

RRC is concerned with:

"How should this UE's radio connection be configured?"

Not:

"Which phone number is Bob?"

Different layer.

9.5 NGAP

Now:

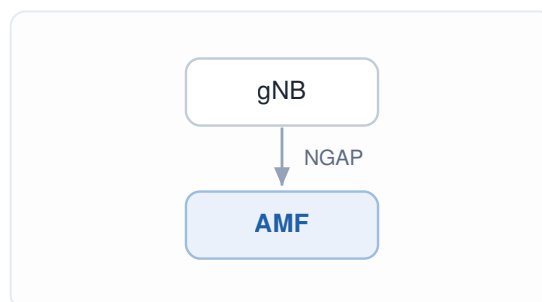


Figure 9.6 gNB → AMF

NGAP is the control-plane protocol between the NG-RAN and 5G Core.

Conceptually:

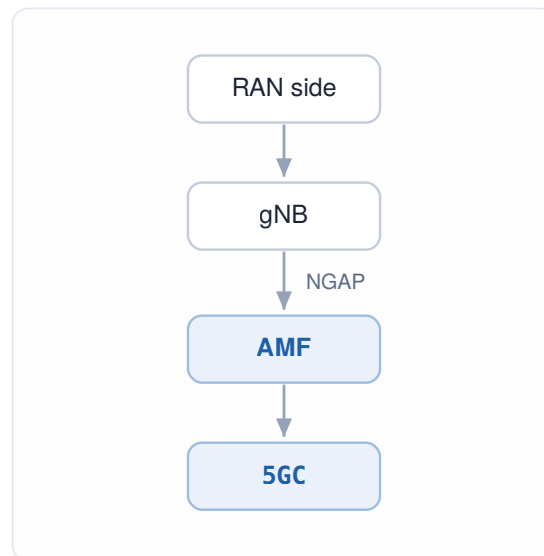


Figure 9.7 RAN side → 5GC

This is where the radio network communicates control information toward the core.

9.6 SCTP

NGAP is transported using:

SCTP

Conceptually:

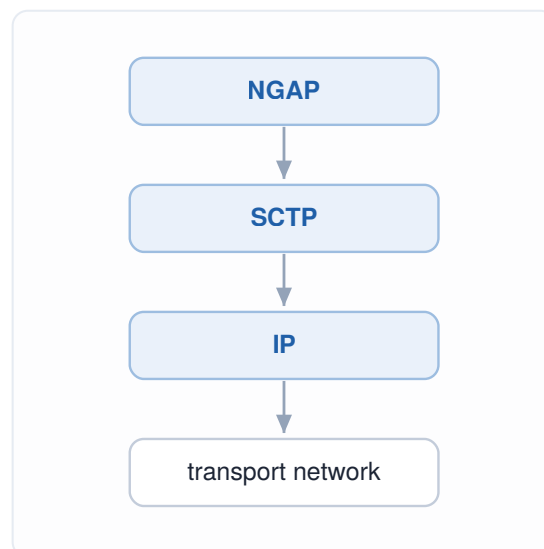


Figure 9.8 NGAP → transport network

So the control stack can look like:

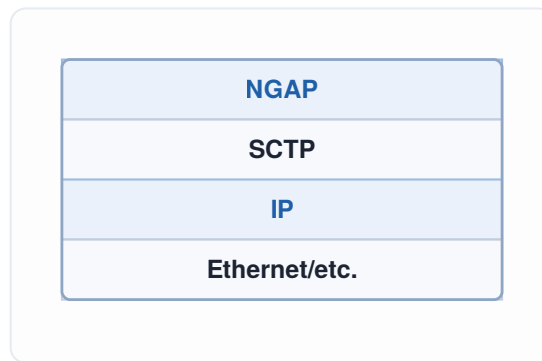


Figure 9.9 Encapsulation stack: NGAP over Ethernet/etc.

Meanwhile user-plane traffic may use:

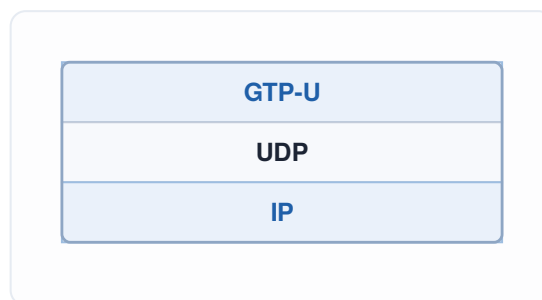


Figure 9.10 Encapsulation stack: GTP-U over IP

Again:

control plane $\neq$ user plane

Service-Based Architecture and HTTP/2

10.1 The 5G core itself is service-oriented

10.2 Where HTTP/2 appears

10.1 The 5G core itself is service-oriented

This is another major difference from old telecom.

5G Core uses a Service-Based Architecture (SBA).

Instead of everything communicating through giant monolithic switches:

giant MSC

5GC uses network functions that expose services.

Conceptually:

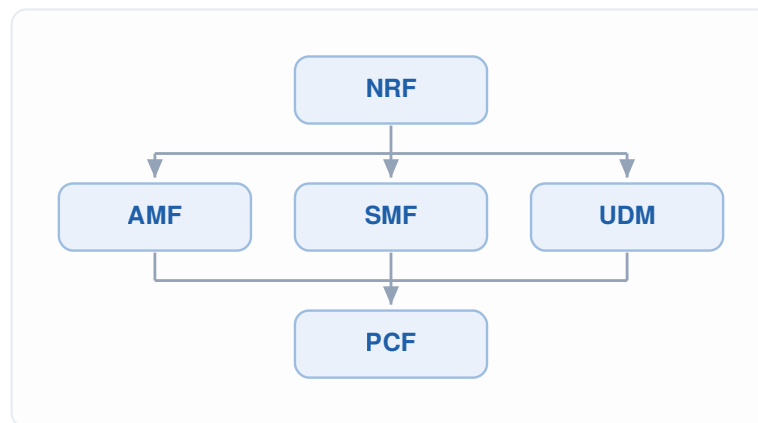


Figure 10.1 NRF → PCF

Communication between many 5GC functions uses service-based APIs, commonly HTTP/2-based.

This is telecom gradually adopting architectural ideas that cloud engineers have been arguing about while pretending microservices are a new invention.

10.2 Where HTTP/2 appears

A simplified example:

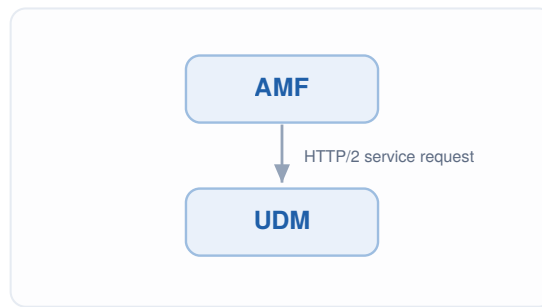


Figure 10.2 AMF → UDM

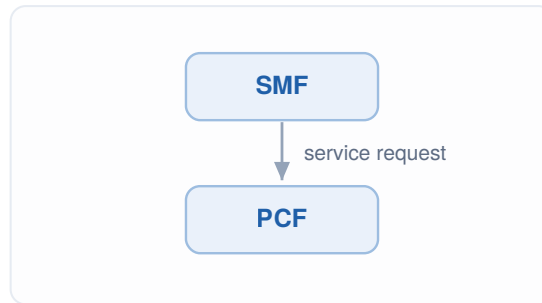


Figure 10.3 SMF → PCF

The exact service and API depend on the network function.

So a modern call can involve:

SIP
HTTP/2
NGAP
SCTP
GTP-U
UDP
RTP

simultaneously.

That sounds absurd until you realize each is solving a different problem.

Comparing the Layers

11.1 Now compare the layers

11.2 Notice the progressive loss of meaning

11.1 Now compare the layers

Let's build the hierarchy:

LAYER	EXAMPLE	QUESTION ANSWERED
Human	+91...	Who am I calling?
Numbering	MSISDN	Which telecom identity?
IMS	SIP URI	Which service/session endpoint?
SIP	INVITE	What session do we want?
SDP	codec/media description	What media can we exchange?
RTP	SSRC/sequence/timestamp	Which media packet is this?
IP	source/destination IP	Which network endpoints?
GTP-U	TEID	Which mobile user-plane tunnel?
5G QoS	QFI	Which QoS flow?
PDCP/RLC/MAC	radio protocol state	How do we transport it over NR?
PHY	waveform	How do we transmit the bits physically?

Table 11.1

This is the real architecture.

11.2 Notice the progressive loss of meaning

At the top:

+91 98BBBBBB20

has semantic meaning to humans.

At the bottom:

010110101001...

has no concept of:

- Bob
- phone number
- voice
- conversation

It's simply a physical encoding of data.

So the information hierarchy is:

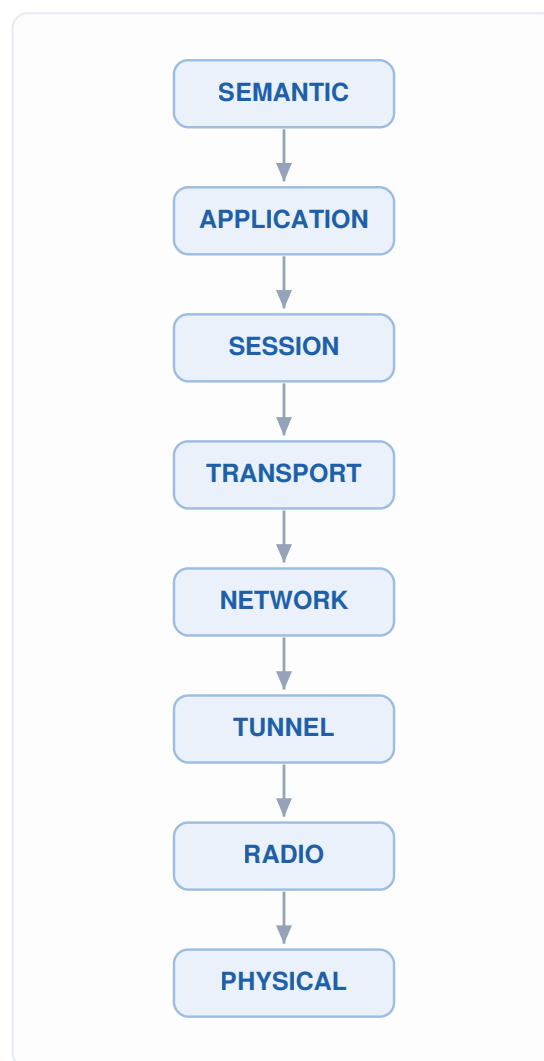


Figure 11.1 SEMANTIC → NETWORK → PHYSICAL

This is the same philosophical idea behind abstraction layers in operating systems and computer architecture.

One Packet's Journey

12.1 One packet's journey

12.2 What the router actually sees

12.3 Why this architecture scales

12.4 And this explains number portability

12.1 One packet's journey

Let's follow a single hypothetical RTP packet.

Alice says part of a syllable.

Step 1

Codec generates:

audio frame

Step 2

RTP:

RTP header

audio frame

Step 3

UDP:

- UDP
- RTP
- audio

Step 4

- IP
- RTP
- UDP
- audio

Step 5

5G user-plane transport:

- GTP-U
- UDP
- audio
- IP
- RTP

Step 6

Radio protocol processing:

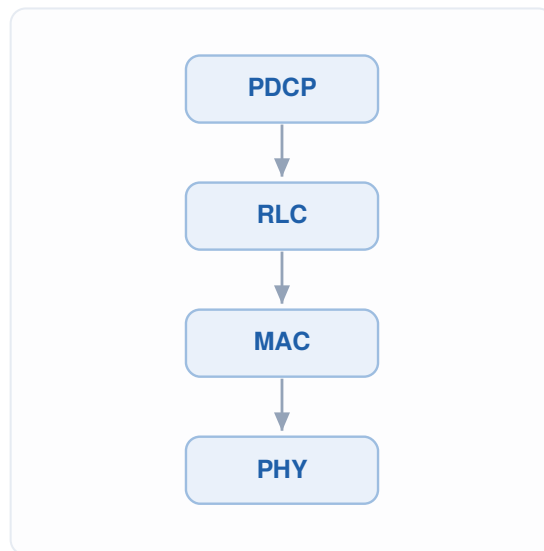


Figure 12.1 PDCP → PHY

Step 7

Electromagnetic transmission:

RF waveform

Step 8

Bob's phone reverses the process:

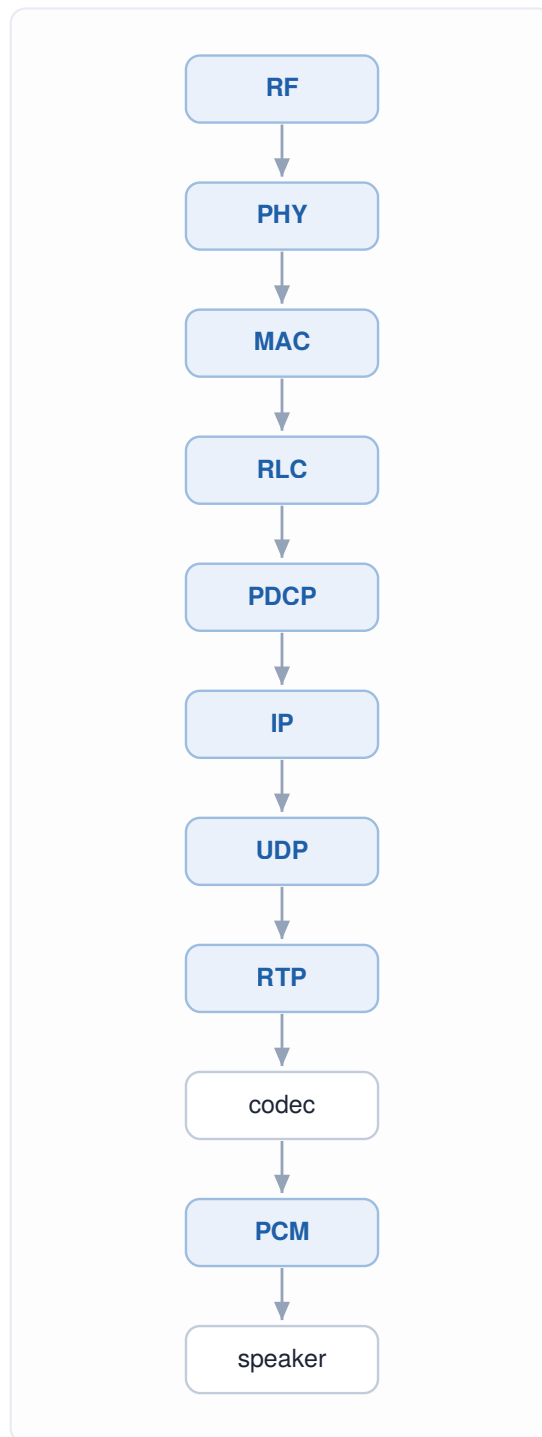


Figure 12.2 RF → IP → speaker

That's encapsulation and decapsulation in action.

12.2 What the router actually sees

An intermediate IP router doesn't care:

"Bob's phone number is +91..."

It sees something more like:

- SRC IP: x.x.x.x
- DST IP: y.y.y.y
- Protocol: UDP

A GTP-U node sees:

GTP-U
TEID

A SIP proxy sees:

- SIP
- From
- To
- Call-ID
- Route

The gNB radio stack sees:

RRC
PDCP
RLC
MAC
PHY

Different devices inspect different layers.

That's the entire point of layering.

12.3 Why this architecture scales

Imagine 1 billion phones.

A centralized machine trying to track:

- phone number
- → physical antenna
- → exact radio frequency

would be catastrophic.

Instead:

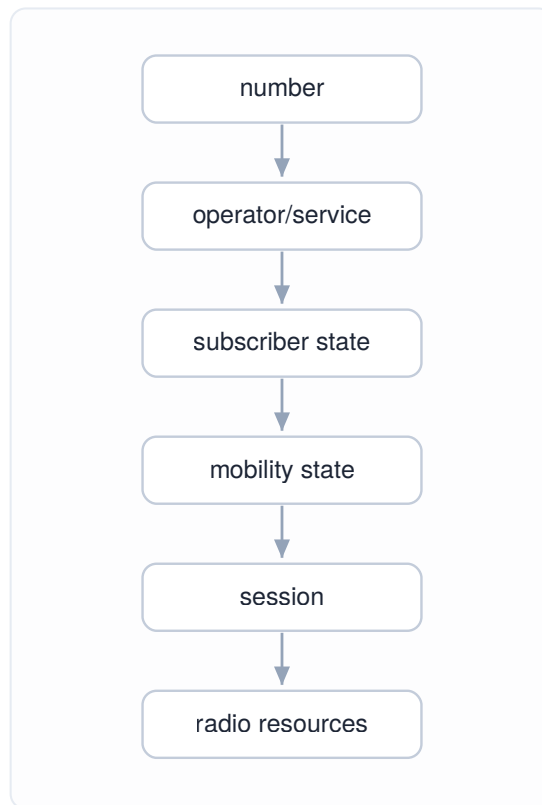


Figure 12.3 number → mobility state → radio resources

Each subsystem maintains the state relevant to its layer.

That's distributed state management.

The phone number doesn't need to know:

- which resource block
- which modulation
- which antenna port
- which HARQ process

The RAN handles that.

The RAN doesn't need to understand:

what Alice's conversation means

The application/service layer handles that.

12.4 And this explains number portability

Now the previous discussion becomes obvious.

Suppose:

+91 98BBBBBB20

was originally Operator A.

After porting:

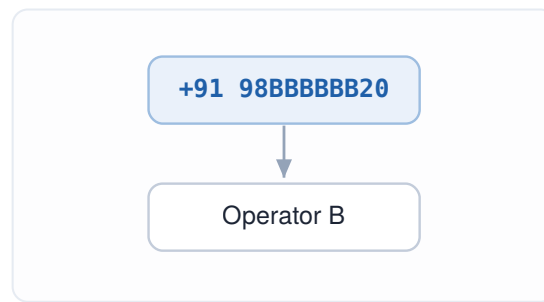


Figure 12.4 +91 98bbbbbb20 → Operator B

The public identifier remained unchanged.

The routing database changed.

Therefore:

- IDENTITY
- LOCATION
- ROUTING
- PHYSICAL ENDPOINT

This is one of the deepest architectural principles in telecommunications.

Reading It in Wireshark

13.1 What you should eventually be able to read in Wireshark

13.2 The packet-capture hierarchy

13.3 And here is the critical security boundary

13.1 What you should eventually be able to read in Wireshark

If you capture a VoIP/IMS trace, you'll encounter things resembling:

- SIP INVITE
- SIP/2.0 100 Trying
- SIP/2.0 183 Session Progress
- SIP/2.0 180 Ringing
- SIP/2.0 200 OK
- ACK
- RTP
- RTP
- RTP
- RTP

RTCP
RTCP

BYE
200 OK

For a 5G radio/core trace you may additionally see:

RRC
NAS
NGAP
HTTP/2
PFCP
GTP-U

The exact visibility depends heavily on where the capture is taken and what encryption/security is active.

13.2 The packet-capture hierarchy

If you captured at different points, you'd see radically different things.

Capture on application endpoint

SIP
RTP

Capture on IP interface

IP
UDP/TCP
SIP/RTP

Capture on 5G transport network

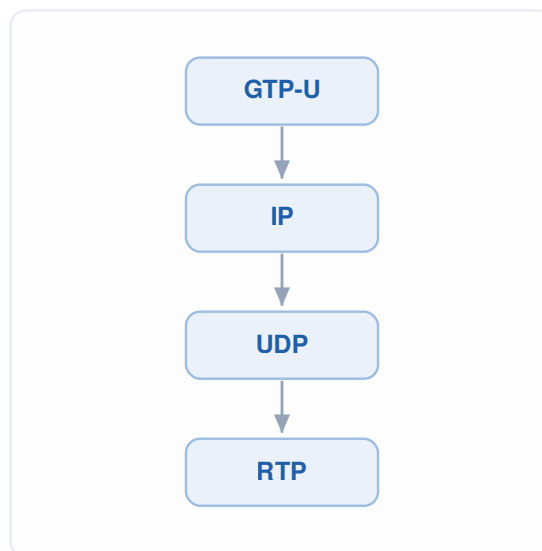


Figure 13.1 GTP-U → RTP

Capture on RAN protocol interface

PDCP
RLC
MAC
RRC

RF capture

You don't see:

SIP INVITE

directly.

You see:

complex RF samples

from which specialized equipment can decode the radio protocol if it has the necessary synchronization, configuration and security material.

13.3 And here is the critical security boundary

This also explains why telecom interception isn't equivalent to:

"read the packets"

There are multiple security layers.

Potentially:

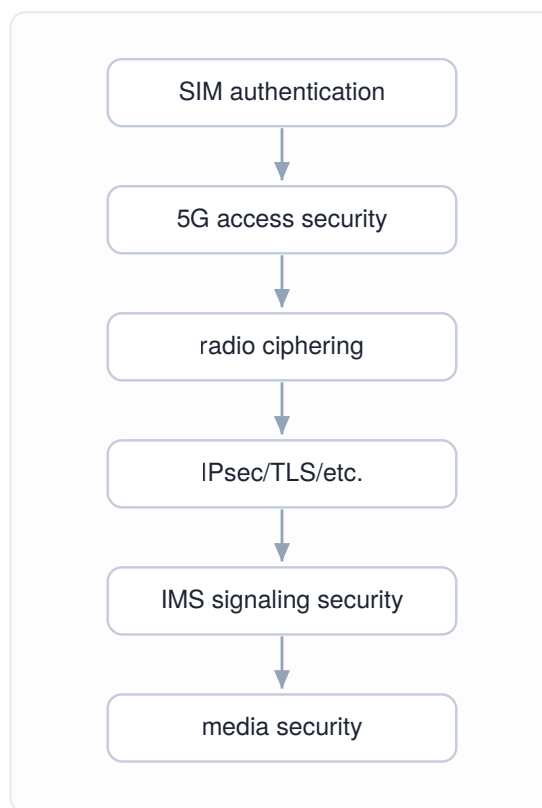


Figure 13.2 SIM authentication → IPsec/TLS/etc → media security

3GPP maintains dedicated security specifications for IMS and media-plane security, including TS 33.328. [3GPP Portal](#)

So seeing a packet does not automatically mean seeing:

Alice's voice

The packet may contain encrypted material.

The Final Mental Model

14.1 The final mental model

14.1 The final mental model

At this point, don't think:

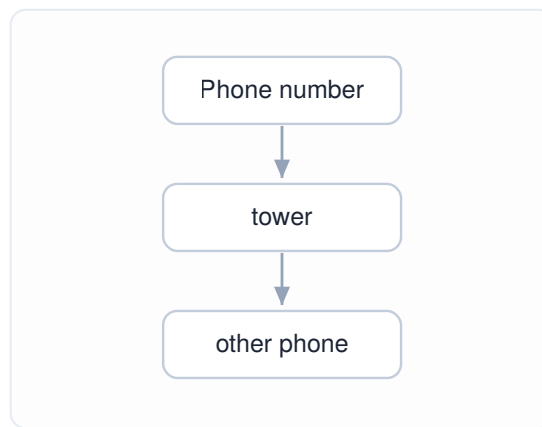


Figure 14.1 Phone number → other phone

That model is fundamentally too primitive.

Think:

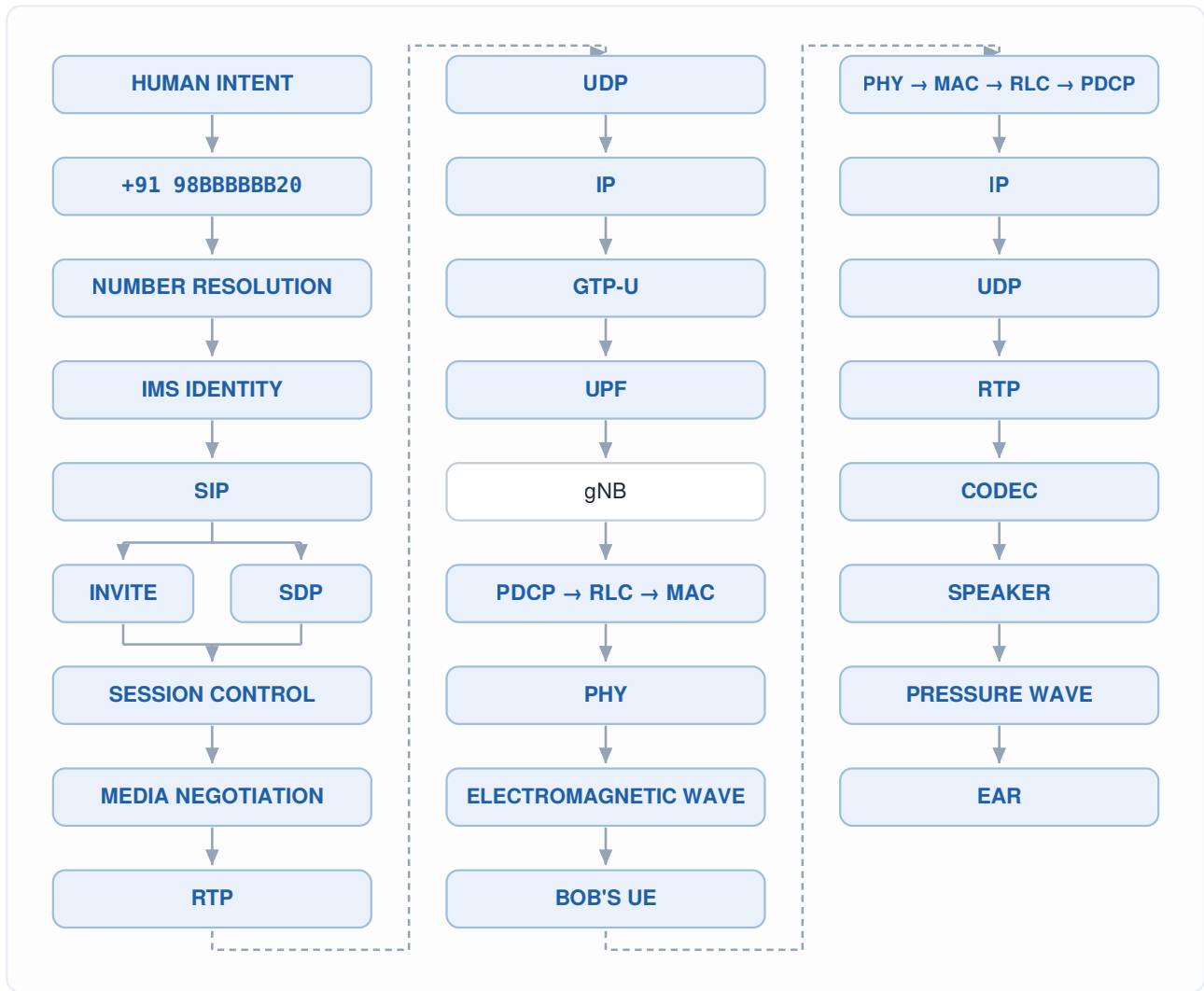


Figure 14.2 Human intent → gNB → EAR

And simultaneously, controlling the whole thing:

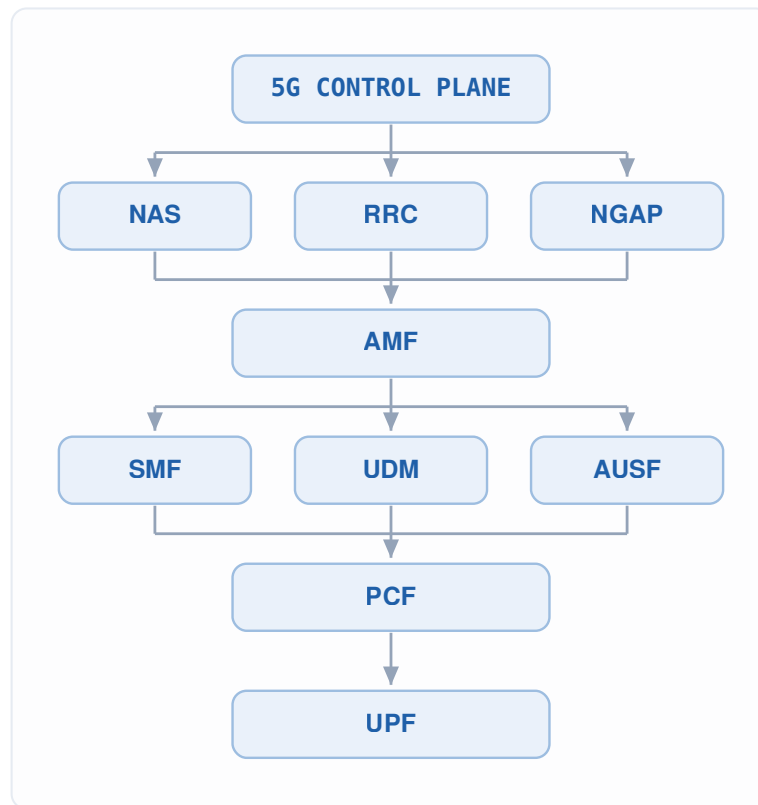


Figure 14.3 5g control plane → Smf / udm / ausf → UPF

This is the point where the phrase "phone call" becomes misleading.

A modern cellular voice call is really a coordinated distributed system involving:

identity resolution + authentication + mobility management + session signaling + service logic + policy control + QoS + packet tunneling + radio scheduling + codec processing + real-time media transport.

The number is merely the first key that gets the whole machine moving.

Standards worth studying directly

Standards worth studying directly

- 3GPP TS 24.229: IMS SIP/SDP call-control protocol. [3GPP Portal](#)
- 3GPP TS 23.228: IMS architecture. [3GPP Portal](#)
- 3GPP TS 38.331: NR RRC.
- 3GPP TS 38.322 / 38.323: NR RLC and PDCP. [3GPP Portal](#)
- RFC 3261: SIP. [RFC Editor](#)
- RFC 4566: SDP. [RFC Editor](#)
- RFC 3550: RTP/RTCP. [RFC Editor](#)

The logical next exercise is to take one real-looking RTP packet and calculate every byte of its header, then wrap it inside UDP → IPv4 → GTP-U, and finally examine how the 5G radio stack segments that data. That gets us from architecture into actual hexadecimal packet anatomy, which is where the fun finally becomes properly unreasonable.